CSC462 – ARTIFICIAL INTELLIGENCE

Lab Assignment 2 Report

Submitted By:
Muhammad Rizwan Shafiq
SP24-BCS-069
BCS-4B

Submitted To:
Dr. Muhammad Imran

Lab 4: Depth First Search (DFS)

Objective: Implement and analyze the Depth First Search algorithm to explore graphs and solve search problems such as route finding and word discovery on a grid.

Task 1: Romania Map Pathfinding using DFS

This program implements a DFS traversal on the Romania city graph to find a path from Arad to Bucharest. The traversal continues until the destination is found, reconstructing the discovered path.

Code Snippet:

```
from typing import Dict, List, Tuple, Set

Graph = Dict[str, List[Tuple[str, int]]]

romania_graph: Graph = {
    "Arad": [("Zerind", 75), ("Sibiu", 140), ("Timisoara", 118)],
    "Zerind": [("Arad", 75), ("Oradea", 71)],
    "Oradea": [("Zerind", 71), ("Sibiu", 151)],
    "Timisoara": [("Arad", 118), ("Lugoj", 111)],
    "Lugoj": [("Timisoara", 111), ("Mehadia", 70)],
    "Mehadia": [("Lugoj", 70), ("Drobeta", 75)],
    "Drobeta": [("Mehadia", 75), ("Craiova", 120)],
    "Craiova": [("Drobeta", 120), ("Rimnicu Vilcea", 146), ("Pitesti",
138)],
    "Sibiu": [("Arad", 140), ("Oradea", 151), ("Fagaras", 99), ("Rimnicu
Vilcea", 80)],
    "Fagaras": [("Sibiu", 99), ("Bucharest", 211)],
    "Rimnicu Vilcea": [("Sibiu", 80), ("Craiova", 146), ("Pitesti", 97)],
    "Pitesti": [("Rimnicu Vilcea", 97), ("Craiova", 138), ("Bucharest",
101)],
    "Bucharest": [("Fagaras", 211), ("Pitesti", 101), ("Giurgiu", 90),
("Urziceni", 85)],
    "Giurgiu": [("Bucharest", 90)],
    "Urziceni": [("Bucharest", 85), ("Hirsova", 98), ("Vaslui", 142)],
    "Hirsova": [("Urziceni", 98), ("Eforie", 86)],
    "Eforie": [("Hirsova", 86)],
    "Vaslui": [("Urziceni", 142), ("Iasi", 92)],
    "Iasi": [("Vaslui", 92), ("Neamt", 87)],
    "Neamt": [("Iasi", 87)],
}

def dfs_path(graph: Graph, start: str, goal: str) -> Tuple[List[str], int,
List[str]]:
    visited: Set[str] = set()
```

```
    visit_order: List[str] = []
    path: List[str] = []
    total_cost = 0

    # Deterministic neighbor order
    sorted_graph: Graph = {u: sorted(v, key=lambda x: x[0]) for u, v in
graph.items()}
    # sorted_graph = graph

    def dfs(u: str) -> bool:
        nonlocal total_cost
        visited.add(u)
        visit_order.append(u)
        path.append(u)
        if u == goal:
            return True
        for v, w in sorted_graph[u]:
            if v not in visited:
                total_cost += w
                if dfs(v):
                    return True
                # backtrack
                total_cost -= w
                path.pop()
        return False

    found = dfs(start)
    if not found:
        return [], 0, visit_order
    return path, total_cost, visit_order

if __name__ == "__main__":
    path, cost, order = dfs_path(romania_graph, "Arad", "Bucharest")
    print("DFS visit order:")
    print(" -> ".join(order))
    print("\nPath from Arad to Bucharest (DFS first-found):")
    print(" -> ".join(path) if path else "No path found")
    print(f"Total distance along this DFS path: {cost}")
```

Expected Output:
DFS path (first-found): Arad → Sibiu → Fagaras → Bucharest
Total distance on that path: 450 km

Task 2: DFS Boggle Word Finder

This task implements a Boggle solver using DFS traversal with backtracking to discover all valid words on a 4x4 grid. The algorithm avoids revisiting cells and prunes paths that cannot form valid dictionary prefixes.

Code:

```
from typing import List, Set, Tuple

# 8-direction movements
DIRECTIONS: List[Tuple[int, int]] = [
    (-1,  0), (1, 0),  (0, -1), (0, 1), # N, S, W, E
    (-1, -1), (-1, 1), (1, -1), (1, 1), # NW, NE, SW, SE
]

def all_prefixes(words: List[str]) -> Set[str]:
    prefixes: Set[str] = set()
    for word in words:
        for i in range(1, len(word) + 1):
            prefixes.add(word[:i])
    return prefixes

def find_boggle_words(board: List[List[str]], dictionary: List[str]) ->
Set[str]:
    if not board or not board[0] or not dictionary:
        return set()

    normalized_board = [[ch.upper() for ch in row] for row in board]
    words_upper = [w.upper() for w in dictionary]
    words_lookup: Set[str] = set(words_upper)
    prefix_lookup: Set[str] = all_prefixes(words_upper)

    num_rows, num_cols = len(normalized_board), len(normalized_board[0])
    used_in_path = [[False] * num_cols for _ in range(num_rows)]
    found_words: Set[str] = set()

    def explore_from_cell(row: int, col: int, current_word: str) -> None:
        next_word = current_word + normalized_board[row][col]

        if next_word not in prefix_lookup:
            return

        if next_word in words_lookup:
            found_words.add(next_word)
```

```
        used_in_path[row][col] = True
        for dr, dc in DIRECTIONS:
            nr, nc = row + dr, col + dc
            if 0 <= nr < num_rows and 0 <= nc < num_cols and not
used_in_path[nr][nc]:
                explore_from_cell(nr, nc, next_word)
        used_in_path[row][col] = False  # backtrack

    for r in range(num_rows):
        for c in range(num_cols):
            explore_from_cell(r, c, "")

    return found_words

if __name__ == "__main__":
    board = [
        list("MSEF"),
        list("RATD"),
        list("LONE"),
        list("KAFB"),
    ]
    dictionary = ["START", "NOTE", "SAND", "STONED"]

    results = find_boggle_words(board, dictionary)
    print("Found words:", sorted(results))
```

Expected Output:
Words found: ['NOTE', 'SAND', 'STONED']

Lab 5: Iterative Deepening Depth First Search (IDFS)

Objective: Combine the advantages of DFS (low memory use) and BFS (completeness) by gradually increasing the search depth until the goal is found, ensuring both space efficiency and completeness.

Task 1: IDFS Pathfinding on Romania Map

This task applies the IDDFS algorithm to the Romania map, searching incrementally deeper to find a route from Arad to Bucharest while minimizing the number of edges explored.

Code:

```
ROMANIA = {
    "Arad": {"Zerind": 75, "Sibiu": 140, "Timisoara": 118},
    "Zerind": {"Arad": 75, "Oradea": 71},
    "Oradea": {"Zerind": 71, "Sibiu": 151},
    "Timisoara": {"Arad": 118, "Lugoj": 111},
    "Lugoj": {"Timisoara": 111, "Mehadia": 70},
    "Mehadia": {"Lugoj": 70, "Drobeta": 75},
    "Drobeta": {"Mehadia": 75, "Craiova": 120},
    "Craiova": {"Drobeta": 120, "Rimnicu Vilcea": 146, "Pitesti": 138},
    "Sibiu": {"Arad": 140, "Oradea": 151, "Fagaras": 99, "Rimnicu Vilcea":
80},
    "Fagaras": {"Sibiu": 99, "Bucharest": 211},
    "Rimnicu Vilcea": {"Sibiu": 80, "Pitesti": 97, "Craiova": 146},
    "Pitesti": {"Rimnicu Vilcea": 97, "Craiova": 138, "Bucharest": 101},
    "Bucharest": {"Fagaras": 211, "Pitesti": 101, "Giurgiu": 90,
"Urziceni": 85},
    "Giurgiu": {"Bucharest": 90},
    "Urziceni": {"Bucharest": 85, "Hirsova": 98, "Vaslui": 142},
    "Hirsova": {"Urziceni": 98, "Eforie": 86},
    "Eforie": {"Hirsova": 86},
    "Vaslui": {"Urziceni": 142, "Iasi": 92},
    "Iasi": {"Vaslui": 92, "Neamt": 87},
    "Neamt": {"Iasi": 87},
}

def ida_star_min_cost(start: str, goal: str, graph=ROMANIA):

    def h(_city: str) -> int:
        return 0

    # Depth-first search bounded by f = g + h <= bound
    def dfs(path: list[str], g: int, bound: int):
        city = path[-1]
        f = g + h(city)
```

```
        if f > bound:
            return None, f
        if city == goal:
            return (path[:], g), f

        min_excess = float("inf")
        for nxt, w in graph[city].items():
            if nxt in path:  # avoid cycles
                continue
            res, t = dfs(path + [nxt], g + w, bound)
            if res is not None:
                return res, t
            if t < min_excess:
                min_excess = t
        return None, min_excess

    bound = h(start)
    path = [start]
    while True:
        result, t = dfs(path, 0, bound)
        if result is not None:
            return result  # (path, cost)
        if t == float("inf"):
            return None
        bound = t  # next bound = smallest f-cost that exceeded the
previous bound


best = ida_star_min_cost("Arad", "Bucharest")
if best:
    route, dist = best
    print("Optimal route:", " -> ".join(route))
    print("Total distance:", dist, "km")
else:
    print("No route found.")

# ==== ID_DFS ====
def iddfs_edges(start, goal, graph, max_depth=50):
    """Iterative Deepening on EDGE DEPTH (fewest hops). Returns first path
found."""
    def dls(node, goal, limit, path, onpath):
        if node == goal:
            return path[:]
        if limit == 0:
            return None
```

```
        for nxt in graph[node]:
            if nxt in onpath:   # avoid cycles
                continue
            onpath.add(nxt); path.append(nxt)
            res = dls(nxt, goal, limit-1, path, onpath)
            if res: return res
            path.pop(); onpath.remove(nxt)
        return None

    for depth in range(max_depth + 1):
        res = dls(start, goal, depth, [start], {start})
        if res: return res
    return None

def path_cost(path, graph):
    return sum(graph[path[i]][path[i+1]] for i in range(len(path)-1))

path = iddfs_edges("Arad", "Bucharest", ROMANIA)
print("ID-DFS path (fewest edges):", " -> ".join(path))
print("Total km on that path:", path_cost(path, ROMANIA))
```

Expected Output:
ID-DFS path (fewest edges): Arad → Sibiu → Fagaras → Bucharest
Total km on that path: 450

Task 2: Iterative Deepening Boggle Word Finder

This program extends the Boggle solver using iterative deepening search to gradually increase the search depth limit (word length) until all possible words are found. Each iteration explores longer word formations while maintaining prefix pruning for efficiency.

Code:

```
from typing import List, Set, Tuple

# 8 directions: N, S, W, E, NW, NE, SW, SE
DIRECTIONS: List[Tuple[int, int]] = [
    (-1, 0), (1, 0), (0, -1), (0, 1),
    (-1, -1), (-1, 1), (1, -1), (1, 1)
]

def all_prefixes(words: List[str]) -> Set[str]:
    prefixes = set()
```

```
    for w in words:
        for i in range(1, len(w) + 1):
            prefixes.add(w[:i])
    return prefixes

def iterative_deepening_boggle(board: List[List[str]], dictionary:
List[str]) -> Set[str]:
    if not board or not dictionary:
        return set()

    board = [[ch.upper() for ch in row] for row in board]
    dict_upper = [w.upper() for w in dictionary]
    words_lookup = set(dict_upper)
    prefix_lookup = all_prefixes(dict_upper)

    rows, cols = len(board), len(board[0])
    found_words: Set[str] = set()
    visited = [[False] * cols for _ in range(rows)]

    def dfs(r: int, c: int, current: str, depth: int, max_depth: int):
        if depth > max_depth:
            return
        next_word = current + board[r][c]
        if next_word not in prefix_lookup:
            return
        if next_word in words_lookup:
            found_words.add(next_word)

        visited[r][c] = True
        for dr, dc in DIRECTIONS:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and not visited[nr][nc]:
                dfs(nr, nc, next_word, depth + 1, max_depth)
        visited[r][c] = False

    # Iteratively deepen search by increasing max word length
    for max_depth in range(3, 9):
        print(f"Searching words up to length {max_depth}")
        for r in range(rows):
            for c in range(cols):
                dfs(r, c, "", 1, max_depth)

    return found_words
```

```python
if __name__ == "__main__":
    board = [
        list("MSEF"),
        list("RATD"),
        list("LONE"),
        list("KAFB"),
    ]
    dictionary = ["START", "NOTE", "SAND", "STONED"]

    result = iterative_deepening_boggle(board, dictionary)
    print("Words found:", sorted(result))
```

Expected Output:
Words found: ['NOTE', 'SAND', 'STONED']

Conclusion

Through Labs 4 and 5, the differences and trade-offs between DFS and IDFS were explored. DFS offers memory efficiency but can miss optimal paths, whereas IDFS ensures completeness by repeatedly deepening the search depth, combining the strengths of both DFS and BFS.